ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL

FACULTAD DE INGERNIERÍA EN ELECTRICIDAD Y COMPUTACIÓN
INGENIERÍA DE SOFTWARE I
ESPECIFICACIÓN DE PROYECTO SEGUNDA EVALUACIÓN

Objetivos Específicos

1. Organizar el desarrollo de un sistema de software mediante la conformación de equipos de trabajo, considerando roles y responsabilidades.
2. Aplicar normas y buenas prácticas de ingeniería de requerimientos para la generación de especificaciones de requerimientos de software para un sistema de mediana complejidad.

Objetivos de Aprendizaje

2. Diseñar, implementar y evaluar una solución basada en computación para cumplir con un conjunto dado de requisitos de computación en el contexto de la disciplina del programa.

Prerrequisito

1. **El documento de especificación de requerimientos completo** del sistema de software aprobado para la asignatura. Este prerrequisito (**incluyendo el prototipo de alta fidelidad** [9]) debe constar entre los entregables. La evaluación del prerrequisito **se constituye en un factor entre 0 y 1 en la evaluación de este subsiguiente avance del proyecto**.

Descripción

El documento debe ser elaborado por un equipo de hasta 4 estudiantes y debe contener los elementos descritos en las secciones que se presentan a continuación. **En todo momento, el diseño del sistema debe ser consistente con el documento de especificación de requerimientos y el prototipo del sistema**.
La aceptación del contenido del documento por parte del cliente debe constar en un acta de aceptación debidamente firmada.

Sección A

Se debe documentar la gestión de riesgos y el cronograma para el proyecto.

Sección B

Modelamiento de la parte estática del sistema utilizando los siguientes diagramas UML:

a. Diagramas de casos de uso y **documentación completa de los 10 casos de uso** más relevantes.
b. Diagramas de clases de **todas las entidades** correspondiente a la **lógica de negocio del sistema**.
c. Diagramas de objetos para aspectos **medulares** del sistema.
d. Diagramas de componentes del sistema.
e. Diagrama de despliegue del sistema.

Sección C

Modelamiento del comportamiento del sistema utilizando los siguientes diagramas UML:

a. Diagramas de actividad de **todos los procesos** del sistema.
b. Diagramas de secuencia de al menos **30 algoritmos transaccionales** relevantes.
c. Diagramas de colaboración/comunicación con fines ilustrativos.
d. Diagramas de estado de **al menos 2 objetos relevantes** en el sistema.

Entregables

Como plantilla del documento, considere la plantilla LaTeX y guías especificadas en la referencia “Writing guidelines and templates” [10] de ETS pero utilizando el logo de ESPOL.

1. El reporte del avance en el desarrollo del sistema de software (**en formato pdf**).
   a. Debe incluir una lista de los integrantes del equipo en la primera página y tabla de contenido, índice de tablas e índice de figuras en las subsiguientes páginas.

b. Diseño de alto nivel y modelos legibles del sistema de software haciendo uso de los diagramas UML vistos en clase. Cada diagrama debe estar en una página y la imagen debe tener buena resolución. Indicar la herramienta en la que se creó los modelos del sistema.
c. Puede incluir cualquier información que soporte su trabajo con relación a los elementos referidos en la rúbrica de evaluación.
d. Cronograma y documentación de riesgos.
e. Un acta de conformidad con el contenido firmado por el representante del cliente.

2. El documento de especificación de requerimientos que al igual que el parcial anterior incluya:
   a. Un prototipo de alta fidelidad del sistema de software aprobado. Adicionalmente, capturas de pantalla del prototipo y su flujo de ventanas.
   b. Un acta de conformidad con el contenido firmado por el representante del cliente.
3. El proyecto correspondiente a la herramienta en la cual se modeló el diseño del sistema (por ejemplo, Visual Paradigm).
4. Cada estudiante debe enviar la co-evaluación del desempeño de los otros integrantes del equipo de trabajo completando la **tarea individual** "Co-evaluación Proyecto".

Rúbrica de Calificación

Descripción	**Valor**
Sección A	
Documentación de riesgos	10
Estructura y contenido de las secciones del documento (incluir cronograma)	10
Sección B	
Diagramas de clases (SOLID, patrones de diseño, malos olores, etc.)	20
Diagramas y documentación de casos de uso	10
Diagramas de objetos	03
Diagramas de componentes	05
Diagramas de despliegue	05
Sección C	
Diagramas de actividad	15
Diagramas de secuencia	15
Diagramas de estado	05
Diagramas de colaboración/comunicación	02
Total	100
Extras	
Definición arquitectónica por medio de ADL [11] (Architecture Description Language) (máx.: +5)	05
Penalidades	
Ausencia de acta de aceptación firmada por el cliente	-100
No subir un entregable de acuerdo con lo requerido (máximo -30)	-10
Cada error gramatical u ortográfico (penalidad máxima: -20)	-02
Cada error de notación UML (penalidad máxima: -30)	-03
Organización de elementos que dificulte la legibilidad de un diagrama (penalidad máxima: -50)	-10
No subir coevaluación en la correspondiente tarea individual (afectación individual)	-100

Late Submission Policy

Delay (§)	Penalty (Ω)
1 hour or less	loss of 10%
1 to 12 hours	loss of 20%
12 to 48 hours	loss of 30%
Over 48 hours:	loss of 100%

(§) every clock hour counts including weekends or holidays
(Ω) automatic and non-negotiable penalty

Referencias

1. Robertson S. and Robertson J, Mastering the Requirements Process: Getting Requirements Right
2. Pressman and Maxim, “Software Engineering”
3. Ian Somerville, “Software Engineering”
4. IEEE Recommended Practice for Software Requirements Specifications
5. Systems and software engineering - Life cycle processes – Requirements engineering
6. The IEEE Guide to the Software Engineering Body of Knowledge – SWEBOK
7. MockupScreens, http://www.mockupscreens.com
8. Perdita Stevens, Using UML
9. Jaye Hannah, “What’s the Difference Between a Wireframe, a Prototype, and a Mockup?”, https://careerfoundry.com/en/blog/ux-design/difference-between-wireframes-prototypes-mockups/
10. ETS, “Writing guidelines and templates”, https://www.etsmtl.ca/en/studies/Guidelines-and-templates
11. Medvidovic, N., & Taylor, R. N. (2000). A classification and comparison framework for software architecture description languages. IEEE Transactions on software engineering, 26(1), 70-93.
12. Visual Paradigm, “Learning Guides”, https://www.visual-paradigm.com/guide/
13. Kirill Fakhroutdinov, “The Unified Modeling Language”, https://www.uml-diagrams.org
14. Bizzdesign, “UML modeling”, https://support.bizzdesign.com/display/knowledge/UML+modeling
15. Common Weakness Enumeration, “Bad Coding Practices”, https://cwe.mitre.org/data/definitions/1006.html
16. Dzone, “9 Best Practices to Handle Exceptions in Java”, https://dzone.com/articles/9-best-practices-to-handle-exceptions-in-java
17. Baeldung, “Exception Handling in Java”, https://www.baeldung.com/java-exceptions